

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Data Platform for Real-Time Anomaly Detection and Intelligent Alert Delivery in Stock Trading Systems

VU THAI HUNG

Hung.VT226046@sis.hust.edu.vn

Major : Information Technology

Thesis advisor : Prof. Tran Viet Trung

Signature of advisor

Department : Department of Information Technology

Institute : School of Information and Communication Technology

Hanoi, 5-2026

Acknowledgments

My student life has passed by brilliantly and full of memories as a part of the School of Information and Communication Technology – Hanoi University of Science and Technology. The process of learning and growing in the environment of HUST has equipped me with valuable skills and the ability to adapt well under pressure. First, I would like to express my love and deep gratitude to my family, who have always been by my side throughout my university years. Equally important, I want to express my sincere thanks to my aunt, uncle, and two wonderful older brothers who accompanied me during the years away from home and gave me the warm feeling of a family. I would like to thank Dr. Tran Viet Trung for his dedicated guidance and support throughout the process of developing my graduation thesis. I am also grateful to all the lecturers at Hanoi University of Science and Technology who have wholeheartedly taught and passed on valuable knowledge to me during my student years. I would also like to thank my colleagues, especially my two managers, for their support and encouragement during this recent period. Lastly, I want to express my heartfelt thanks to the friends who shared with me a wonderful chapter of student life.

Abstract

In today's financial markets, vast volumes of trading data are generated every second across thousands of equities. Detecting unusual price and volume activity in real time is a valuable capability for investors and risk managers, yet existing tools either carry prohibitive costs or lack the contextual explanation needed to act on an alert confidently. Building a system that monitors markets continuously, filters out noise, and delivers meaningful notifications remains an open engineering challenge.

This thesis presents a real-time stock anomaly detection platform covering 50 US equities. The system ingests live market data from multiple external sources, applies a statistical rule-based engine to identify abnormal price and volume behaviour, and notifies users through a Telegram bot with sufficient context to support decision-making. Users can also define their own alert conditions directly through bot commands, without any additional infrastructure.

The underlying data platform follows a layered architecture separating concerns across ingestion, storage, and processing. A streaming layer handles real-time market data as it arrives, while a medallion lakehouse — comprising Bronze, Silver, and Gold tiers — provides structured long-term storage optimised for analytical queries. Scheduled batch jobs refresh statistical baselines nightly and synchronise operational records to the analytical layer. An orchestration layer automates and monitors the full pipeline schedule.

The final system delivers an end-to-end, modular data pipeline that covers real-time ingestion, stream processing, batch analytics, operational storage, and user-facing alerting — demonstrating how modern open-source big data technologies can be composed into a cohesive, production-ready platform for financial market surveillance.

Contents

Abstract	ii
List of Figures	v
List of Tables	vi
List of Acronyms	viii
List of Notations	ix
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives and Scope	2
1.3 Proposed Solution	2
1.4 Thesis Organisation	3
2 Requirement Analysis	5
2.1 Status Survey	5
2.2 Functional Requirements	7
2.3 Non-functional Requirements	9
2.3.1 Scalability and Performance	9
2.3.2 Fault Tolerance and Data Consistency	9
2.3.3 Security and Access Control	10
3 Technologies Used	11
3.1 Apache Kafka	11
3.2 Apache Spark	12
3.2.1 Spark Structured Streaming	12
3.2.2 Spark Batch Processing	13
3.2.3 Deployment on Kubernetes	13

3.3	Apache Airflow	14
3.3.1	Architecture	14
3.3.2	Role in the Platform	14
3.4	Apache Iceberg	14
3.5	Apache Gravitino	15
3.6	MinIO	15
3.7	Trino	16
3.8	PostgreSQL	16
3.9	FastStream	17
3.10	Keycloak	17
3.11	Kubernetes	18
4	System Design and Deployment	19
4.1	System Design	19
4.1.1	Overall Architecture	19
4.1.2	Data Ingestion Module	21
4.1.3	Data Storage Module	23
4.1.4	Data Processing Module	25
4.1.5	Workflow Orchestration Module	26
4.1.6	Event Delivery Module	27
4.2	System Deployment	28
4.2.1	Data Ingestion Module	29
4.2.2	Data Storage Module	30
4.2.3	Data Processing Module	31
4.2.4	Workflow Orchestration Module	32
4.2.5	Event Delivery Module	33
4.2.6	System Implementation Result	34
5	Analytical Problems And Solutions	37
6	Experimental Results	38
7	Conclusion and Future Work	39
	References	40
	Appendices	41
A	This is title of appendix A	41

List of Figures

2.2.1 Functional Decomposition Model of the proposed platform.	8
4.1.1 Overall system architecture.	20
4.1.2 Yahoo Finance quote producer.	22
4.1.3 Kafka UI: topic raw.stock.quotes.	23
4.1.4 Data storage module architecture.	24
4.1.5 Kafka-to-Iceberg flow of the stream processing jobs.	25
4.1.6 Daily OHLCV pipeline DAG with TimeSensor gates.	27
4.1.7 Alert Service fan-out delivery pipeline.	27
4.2.1 Single-node Kubernetes cluster running on Docker Desktop.	29
4.2.2 Kafka KRaft cluster Kubernetes deployment layout.	30
4.2.3 Data Storage Module Kubernetes deployment layout.	31
4.2.4 SparkApplication submission and lifecycle on Kubernetes.	32
4.2.5 Airflow CeleryExecutor deployment on Kubernetes.	33
4.2.6 Event Delivery Module Kubernetes deployment layout.	34
4.2.7 Single control-plane node in the Kubernetes cluster.	34
4.2.8 Kubernetes workload overview for the stock-anomaly-detection names- pace.	35
4.2.9 Running pods in the stock-anomaly-detection namespace.	35
4.2.10Deployments in the stock-anomaly-detection namespace.	36
4.2.11ClusterIP services in the stock-anomaly-detection namespace.	36
4.2.12Persistent Volume Claims bound in the stock-anomaly-detection names- pace.	36

List of Tables

2.1.1 Comparison of existing market surveillance tools.	6
2.1.2 Comparison of data pipeline architectures for market surveillance.	7
4.1.1 Daily OHLCV pipeline job schedule.	26

List of Acronyms

DL deep learning. vii

IA intelligent agent. vii

MDP Markov decision process. vii

QoS Quality of Service. vii

RL reinforcement learning. vii

WRSN wireless rechargeable sensor network. vii

WSN wireless sensor network. vii

List of Notations

$\mathcal{P}$ a set of deployed sensors.

$\tilde{E}_{td}$ energy requesting threshold.

n number of deployed sensors.

p_0 base station.

p a sensor.

Chapter 1

Introduction

1.1 Problem Statement

Modern financial markets operate at a scale that makes manual monitoring impractical. On a typical trading day, the US equity market alone processes billions of transactions across thousands of listed securities, with price and volume conditions changing within milliseconds. Within this volume of activity, a small fraction of events represent genuine anomalies — sudden deviations in price or trading volume that cannot be explained by ordinary market dynamics. These events carry significant information value for investors: they may signal undisclosed corporate news, large institutional order flow, or emerging market stress before the situation becomes widely known.

Despite this opportunity, the tools available to non-institutional participants are inadequate. High-end platforms such as Bloomberg Terminal provide comprehensive market surveillance but are prohibitively expensive for individual investors and small funds. Lower-cost screeners, such as Finviz’s unusual volume scanner, offer limited rule sets and no mechanism for contextual explanation — a user receives a numerical alert but has no way to determine whether the anomaly reflects a real event or a data artefact. The gap between raw signal and actionable insight forces investors to spend hours on manual research for each alert received.

A further challenge is the data engineering complexity involved in building a real-time market surveillance system. Live quote feeds, historical OHLCV data, news articles, and company metadata must be ingested from heterogeneous sources, cleaned, and made available to detection logic within latency budgets measured in single-digit milliseconds per quote. Persisting this data for long-term analytics while simultaneously serving a real-time detection workload requires careful architectural separation between streaming and

batch processing layers. These requirements place such a system beyond the reach of simple scripted solutions.

1.2 Objectives and Scope

The primary objective of this thesis is to design and implement a production-grade, real-time anomaly detection platform for US equities that addresses the limitations described above. Specifically, the system must:

1. Continuously ingest live market data for 500 US stocks from multiple external data sources without data loss.
2. Apply a statistical rule-based detection engine to identify abnormal price and volume behaviour on each incoming quote, with end-to-end latency under 10 milliseconds per event.
3. Deliver alerts to users via Telegram with sufficient context to support immediate decision-making, distinguishing between anomalies of different severity levels.
4. Allow users to define personalised alert rules through Telegram Bot commands, without requiring additional infrastructure or developer intervention.
5. Persist all market data in a structured analytical store to support historical querying and reporting.

The scope of this project covers 500 actively traded US equities drawn from the S&P 500 index. The detection layer applies six statistical rules derived from z-score analysis, Bollinger Band theory, and relative strength metrics, calibrated against a rolling 20-day historical baseline. The data platform covers ingestion, stream processing, batch transformation, and long-term storage. Real-time notification and user-defined alert management via a Telegram Bot are included. This thesis does not cover portfolio management, order execution, or integration with brokerage APIs.

1.3 Proposed Solution

The proposed solution is a multi-layer data platform composed of loosely coupled services, each responsible for a distinct concern in the pipeline.

Data ingestion. Live quote ticks for 500 symbols are collected via the Yahoo Finance WebSocket and published to a message broker. Intraday trade data arrives through the Finnhub WebSocket API. Company news is polled from the Finnhub REST API on a

fixed interval and routed to the same broker. Daily OHLCV history is loaded from Yahoo Finance in a nightly batch job. This separation ensures that real-time and batch workloads draw from independent data paths and do not interfere with each other.

Rule-based detection. An asynchronous microservice consumes the live quote stream and evaluates six statistical rules on each tick: price z-score, volume z-score, volume-to-20-day-average ratio, Bollinger Band position, 14-day RSI extreme, and intraday price range. Each rule is parameterised against a pre-computed context table refreshed nightly by a batch job, keeping detection logic stateless at runtime. Quotes that satisfy one or more rule thresholds are forwarded to an alert delivery service, which formats and sends the notification to the user’s Telegram account.

User-defined alerts. A Telegram Bot allows users to register custom alert rules via simple text commands, specifying a symbol, a market field, a comparison operator, and a threshold. Rules are persisted in an operational database and hot-reloaded into the detection service without a restart. Alert history is recorded in the same database and synchronised to the analytical layer by a daily batch job.

Data platform. All persisted data is organised in a medallion lakehouse architecture comprising three tiers. The Bronze layer stores raw ingested data. The Silver layer provides cleaned and normalised views. The Gold layer implements a star schema optimised for analytical queries, covering daily OHLCV facts, anomaly event facts, and alert history. A dedicated scheduler monitors and triggers the full set of daily batch jobs. This architecture separates streaming and batch workloads cleanly, enables independent scaling of each component, and is built entirely on open-source technologies.

1.4 Thesis Organisation

The remainder of this thesis is organised as follows.

Chapter 2 — Requirement Analysis examines the problem space in detail. It surveys the current state of market surveillance tools and their limitations, then derives the functional requirements of the proposed system, including data ingestion, anomaly detection, and alert delivery. Non-functional requirements covering scalability and performance, fault tolerance and data consistency, and security and access control are also defined.

Chapter 3 — Technologies Used introduces the key technologies selected for the implementation. Each component of the stack is presented with its motivation and role in the overall architecture, covering stream processing, distributed storage, batch computation, workflow orchestration, and the operational database.

Chapter 4 — System Design and Deployment presents the complete architecture of the platform. It describes the design of each service, the message-passing contracts between components, the lakehouse schema, the batch pipeline schedule, and the Kubernetes-based deployment configuration.

Chapter 5 — Real-Time Detection and Alert Delivery focuses on the core value of the system. It covers the design and calibration of the six statistical detection rules, the custom user-defined alert logic including cross-point operators, alert formatting, Telegram delivery, rate limiting, and the key engineering challenges encountered during implementation.

Chapter 6 — Experimental Results presents the evaluation of the implemented system. It reports end-to-end latency measurements, throughput benchmarks under 500-symbol load, and functional test results covering detection accuracy and alert delivery correctness.

Chapter 7 — Conclusion and Future Work summarises the contributions of this thesis, reflects on design decisions and their trade-offs, and proposes directions for future development including LLM-based anomaly validation and expanded symbol coverage.

Chapter 2

Requirement Analysis

This chapter analyses the requirements of the proposed stock anomaly detection platform. Section 2.1 surveys existing market surveillance tools and identifies the gaps that motivate this work. Section 2.2 derives the functional requirements from those findings. Section 2.3 defines the non-functional requirements covering performance, fault tolerance, and security.

2.1 Status Survey

Financial market data exhibits characteristics that make it particularly challenging to process and monitor. In terms of *volume*, the US equity market alone generates billions of trade events per day across thousands of listed securities. In terms of *velocity*, quote prices and volumes update within milliseconds, and any detection system that cannot keep pace with this rate will produce stale or missed alerts. In terms of *variety*, relevant data spans multiple heterogeneous sources: real-time tick feeds, daily OHLCV aggregates, news articles, and company fundamentals, each with different schemas, latencies, and update frequencies. In terms of *veracity*, data quality issues such as stale quotes, erroneous spikes, and source disagreements are common and must be handled without suppressing genuine anomalies.

These four characteristics together demand a system architecture that separates real-time ingestion and detection from historical storage and batch analysis, while maintaining consistency across both paths. The following subsections survey existing tools and architectural patterns to identify the approach best suited to this problem.

Bloomberg Terminal. Bloomberg Terminal is the most comprehensive professional platform for financial market monitoring. It provides real-time data feeds, customisable

alerts, and anomaly screening across global markets. However, its annual subscription cost exceeds \$24,000 per user, placing it entirely out of reach for individual investors and small funds. Furthermore, Bloomberg’s alert system is general-purpose and does not provide automated contextual explanation for why an anomaly occurred — the analyst must still investigate manually.

Finviz Unusual Volume Screener. Finviz offers a web-based equity screener that includes an unusual volume filter, identifying stocks whose intraday volume exceeds a configured multiple of their average. The tool is accessible at low cost and covers the US equity market broadly. However, it has significant limitations: screening is performed on a polling basis rather than in real time, the rule set is fixed and cannot be personalised by the user, and no contextual information is attached to the alert. A user sees that a stock has elevated volume but has no indication of whether the event is news-driven or unexplained.

UnusualVolumeDetector (open source). UnusualVolumeDetector is a Python script published on GitHub with approximately 975 stars and 268 forks. It scans the entire market for volume anomalies by computing a rolling z-score over five months of daily volume history and flagging stocks whose recent volume exceeds ten standard deviations. While this demonstrates that a z-score approach is effective for volume anomaly detection, the implementation is a single-file script with no streaming capability, no persistent data store, no alert delivery mechanism, and no support for user-defined rules. It is suitable for ad hoc analysis but not for continuous, production-grade monitoring.

Summary. Table 2.1.1 summarises the capabilities of existing tools against the requirements of the proposed system. No existing open-source solution combines real-time streaming ingestion, multi-rule statistical detection, contextual alert delivery, and user-defined rule management at production scale.

Table 2.1.1: Comparison of existing market surveillance tools.

Feature	Bloomberg	Finviz	UnusualVolumeDetector	Proposed
Real-time streaming	✓	–	–	✓
Multi-rule detection	✓	Limited	Single	✓
Alert delivery	✓	–	–	✓
Contextual explanation	Partial	–	–	✓
User-defined rules	Partial	–	–	✓
Open source	–	–	✓	✓
Low cost	–	Partial	✓	✓

Beyond the choice of detection tool, a second architectural decision concerns how to store and serve the underlying market data. Three established patterns are relevant: batch-

only pipelines, pure streaming systems, and hybrid lakehouse architectures. Table 2.1.2 compares these approaches across the criteria most relevant to this project.

Table 2.1.2: Comparison of data pipeline architectures for market surveillance.

Criteria	Batch-only	Pure Streaming	Hybrid Lakehouse
Real-time detection latency	High	Low	Low
Historical query support	Good	Limited	Excellent
Data consistency guarantees	Strong	Weak	Strong
Operational complexity	Low	Medium	Medium
Storage cost efficiency	High	Low	High
Schema evolution support	Limited	Limited	Excellent
Suitable for this project	–	Partial	✓

A batch-only architecture, as used by UnusualVolumeDetector, refreshes statistics once per day and cannot deliver sub-second alerts during market hours. A pure streaming architecture can sustain low-latency detection but provides no efficient mechanism for long-term retention, historical backtesting, or analytical reporting across months of data. A hybrid lakehouse architecture addresses both concerns: a streaming layer handles real-time detection while a structured lakehouse handles historical persistence and analytical queries. This is the approach adopted in the proposed system, using a medallion design with Bronze, Silver, and Gold tiers to separate raw ingestion, cleaned data, and analytics-ready aggregations.

2.2 Functional Requirements

The system is decomposed into six functional modules, each responsible for a distinct concern in the overall data pipeline. Figure 2.2.1 presents the functional decomposition model.

The **Data Ingestion Module** acquires market data from all external sources and publishes it to the internal message broker. Three producers run continuously: a quote producer streaming live prices for 500 symbols via the Yahoo Finance WebSocket, a trade tick producer collecting intraday transactions via the Finnhub WebSocket, and a news producer polling the Finnhub REST API every five minutes for company news articles.

The **Stream Processing Module** is the real-time processing core of the pipeline. It loads a pre-computed 20-day statistical context for each symbol at startup and evaluates six statistical rules on every incoming quote tick: price z-score, volume z-score, volume-to-20-day-average ratio, Bollinger Band position, RSI extreme, and intraday price range.

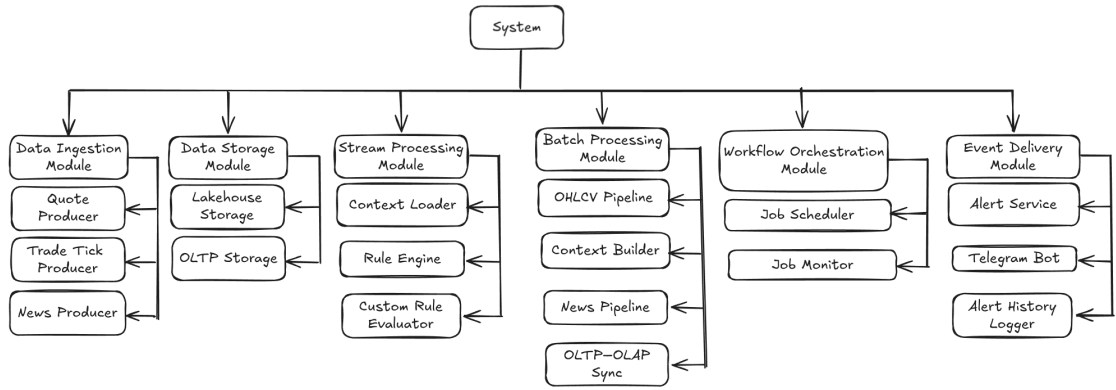


Figure 2.2.1: Functional Decomposition Model of the proposed platform.

A custom rule evaluator runs in parallel, applying any user-defined conditions registered via the bot interface. Candidate anomalies are published downstream for delivery.

The **Event Delivery Module** manages the full notification lifecycle. It formats outgoing alert messages with the triggered symbol, rule, severity level, and relevant metric values; applies per-user rate limiting; and delivers the notification to the recipient’s Telegram account. All delivery events are persisted in the operational store for auditability. The Telegram Bot sub-component additionally handles user registration and custom alert rule management, propagating rule changes to the stream processor via an internal HTTP endpoint without a service restart.

The **Data Storage Module** is divided into two distinct stores serving different workloads. The *Lakehouse* organises all analytical data in a medallion architecture: the Bronze layer retains raw ingested records as immutable Iceberg tables; the Silver layer provides cleaned and deduplicated views; and the Gold layer implements a star schema optimised for analytical queries over OHLCV history, anomaly events, and alert delivery history. The *OLTP Store*, backed by PostgreSQL, manages operational state: user registrations, custom alert rule definitions, alert event logs, and synchronisation watermarks.

The **Batch Processing Module** runs a set of scheduled Spark jobs that promote data through the lakehouse tiers. The OHLCV pipeline ingests daily price history and builds dimension and fact tables in the Gold layer. The news pipeline streams and cleans article data into the Silver layer. The context builder computes rolling 20-day statistics and writes the result to the Gold layer for use by the stream processor. The OLTP–OLAP sync job performs incremental extraction from the PostgreSQL event log and appends new records to the Gold alert history table using a watermark pattern.

The **Workflow Orchestration Module** schedules, monitors, and retries all batch jobs. It enforces the daily execution order — OHLCV ingestion at 06:00 UTC, context refresh

at 07:00 UTC, stream processor hot-reload at 07:15 UTC, and OLTP sync at 07:30 UTC — and surfaces job health to operators through task-level status tracking.

2.3 Non-functional Requirements

In addition to the functional requirements above, the system must satisfy several non-functional requirements to ensure stable and reliable operation in a real-time, data-intensive environment. These requirements cover scalability and performance, fault tolerance and data consistency, and security and access control.

2.3.1 Scalability and Performance

The system is designed to handle a continuous, high-frequency stream of market events across 50 symbols simultaneously. The stream processing layer must evaluate all applicable detection rules on each incoming quote within 10 milliseconds, ensuring that alerts remain actionable by the time they reach the user. During peak market periods — such as market open or major earnings releases — event arrival rates can spike sharply; the message broker is therefore partitioned per symbol group so that burst traffic is absorbed without backpressure on the producers.

The batch processing window between market close (approximately 21:00 UTC) and the next market open (13:30 UTC) must be sufficient to complete all nightly jobs, including OHLCV ingestion, statistical context refresh, and OLTP–lakehouse synchronisation, with margin left for retries. By deploying each service as an independent Kubernetes workload, the platform can scale horizontally — adding replicas to the ingestion producers or stream processor — without affecting other components.

2.3.2 Fault Tolerance and Data Consistency

The system must not lose market events or alert records under partial failure. The message broker is configured with topic replication so that a single broker failure does not cause data loss. The stream processor commits consumer offsets only after a quote has been fully evaluated and any resulting alert published, providing at-least-once processing guarantees.

Data consistency between the OLTP store and the lakehouse is maintained through a watermark pattern: the OLTP–OLAP sync job reads the last successfully committed high-water mark from the `sync_watermarks` table, queries only new alert events beyond that point, and advances the watermark only after the corresponding Iceberg write completes atomically. This ensures that a job crash or retry never produces duplicate or missing

records in the Gold layer. All lakehouse writes use append-only semantics; the Iceberg table format provides snapshot isolation and atomic commits at no additional engineering cost. The workflow orchestrator retries failed batch tasks with exponential backoff and maintains a full execution log for post-incident debugging.

2.3.3 Security and Access Control

All credentials — including data provider API keys, the Telegram Bot token, and database connection strings — are supplied exclusively through environment variables or Kubernetes Secrets and never appear in source code or version-controlled configuration files.

Access to internal infrastructure components — the Iceberg catalog, the workflow orchestrator, and the analytical query engine — is protected by OAuth2/OIDC using Keycloak as the identity provider. Services authenticate via the Authorization Code flow with PKCE for browser-based interfaces, and via client credentials for service-to-service calls. Keycloak issues signed JWT access tokens that each service validates locally against the provider's published JWKS, avoiding a round-trip to the identity server on every request.

Database access follows the principle of least privilege. Each microservice is granted only the permissions it requires on the operational store: the Stream Processing service may read alert rule definitions and append new alert events; the Event Delivery service may read user records and write delivery logs. No service is granted superuser privileges. At the application level, the Telegram Bot authenticates each request by the sender's Telegram chat identifier, ensuring that only the rule owner can modify or delete their own alert rules.

Chapter 3

Technologies Used

This chapter surveys the core technologies on which the platform is built. Each section explains the motivation for choosing a particular technology and describes the specific role it plays within the system. Together, these components form the layered architecture introduced in Chapter ??: a streaming ingestion tier, a medallion lakehouse for long-term storage, a batch-processing engine, an SQL query layer, an OLTP store, and a container-orchestration platform that hosts every workload.

3.1 Apache Kafka

Apache Kafka is a distributed, fault-tolerant event-streaming platform originally developed at LinkedIn and open-sourced under the Apache Software Foundation. Its design centres on a persistent, replicated, partitioned log: producers append records to named *topics*, and consumers read from those topics at their own pace by tracking a numeric *offset* rather than requiring the broker to maintain per-consumer state. This decoupling allows producers and consumers to scale independently and to tolerate temporary failures without losing records.

Kafka organises each topic into one or more partitions. Every record within a partition is ordered; records across different partitions are consumed in parallel, providing horizontal throughput scaling. Replication is configured at the topic level — each partition has a designated *leader* that handles reads and writes, and a configurable number of *followers* that replicate the leader’s log. If the leader fails, one of the followers is elected as the new leader with no data loss, provided the replication factor is at least two.

Consumer groups provide a built-in mechanism for load distribution. When multiple instances of a consumer belong to the same group, Kafka assigns each partition to exactly

one member, ensuring that every record is processed by exactly one instance within the group. When a consumer commits an offset only after a record has been fully processed, the platform delivers at-least-once processing semantics: should a consumer crash before committing, the record will be redelivered on restart.

In this platform, Kafka serves as the central transport layer between all data producers and downstream processing services. Three topics carry market data inbound: `raw.stock.quotes` streams live price ticks published by the Yahoo Finance producer; `raw.stock.trades` carries intraday transaction records from the Finnhub WebSocket producer; and `raw.stock.news` carries news articles polled from the Finnhub REST API. Two alert topics carry data through the detection pipeline: `alerts.raw` receives candidate anomaly events published by the Rule Engine, and `alerts.confirmed` receives events that have passed downstream validation. Each topic is partitioned by symbol so that records belonging to the same security are always routed to the same partition, preserving per-symbol ordering guarantees that the Rule Engine depends upon.

Kafka 4.0.0 is deployed on Kubernetes via the Bitnami Helm chart in *KRaft* mode, replacing the traditional ZooKeeper dependency with an internal Raft-based consensus protocol (KIP-500). The deployment runs a single node in *combined* mode — fulfilling both controller and broker roles in one process — which is sufficient for the platform’s current scale of 50 symbols across five topics. The broker is backed by a persistent volume claim so that topic data and metadata survive pod restarts.

3.2 Apache Spark

Apache Spark is an open-source, distributed data processing framework designed for large-scale analytics workloads. Spark represents computation as a *Resilient Distributed Dataset* (RDD) or, in its higher-level API, as a *DataFrame* or *Dataset* abstracted over a directed acyclic execution plan. The Spark scheduler partitions work into stages and tasks that it distributes across a cluster of *executor* JVM processes, with fault tolerance provided by re-computing lost partitions from the lineage graph rather than by replicating intermediate results.

3.2.1 Spark Structured Streaming

Spark Structured Streaming is the continuous-processing mode of the Spark SQL engine. It models a stream as an unbounded *DataFrame*, applying the same transformations and aggregations that one would use on a static table. Computations execute incrementally: Spark maintains state across micro-batches and updates output tables as new records ar-

rive. Window aggregations over event time require a *watermark* declaration, which informs the engine how long to wait for late-arriving records before finalising an aggregate and releasing state memory.

In this platform, Structured Streaming is used by the `news-ingest-stream` Spark application to continuously read news articles from the `raw.stock.news` Kafka topic and write them to the Bronze layer of the Iceberg lakehouse. Using Spark rather than a lightweight consumer for this path ensures that the Bronze writes inherit Iceberg’s atomic commit and snapshot-isolation guarantees.

3.2.2 Spark Batch Processing

Four Spark batch applications drive the nightly lakehouse refresh. The `ohlcv-daily-loader` downloads historical price data from Yahoo Finance and persists raw records to the Bronze layer. The `news-cleaner` deduplicates and cleans raw news articles and writes the results to Silver. The `rule-engine-context-builder` reads Silver OHLCV records, computes 20-day rolling statistics (mean, standard deviation, Bollinger Bands, RSI, and ATR) for each symbol, and writes the result to a Gold layer table consumed by the Rule Engine at startup. The `fact-ohlcv-daily-builder` promotes cleaned price records into Gold-layer dimension and fact tables organised as a star schema.

All Spark applications in this project are written in Scala 2.12 against Spark 3.5.1 and use the `iceberg-spark-runtime-3.5_2.12` library (version 1.10.1) to read and write Iceberg tables stored on MinIO.

3.2.3 Deployment on Kubernetes

Spark applications are managed on Kubernetes through the *Kubeflow Spark Operator*, deployed via its official Helm chart into the `stock-anomaly-detection` namespace. The operator introduces a `SparkApplication` custom resource definition (CRD): each Spark job is described declaratively as a `SparkApplication` manifest, and the operator’s controller watches for these resources, invokes `spark-submit` internally, and reconciles the actual state of driver and executor pods against the desired specification. A mutating web-hook handles pod injection and annotation, while an RBAC service account grants the driver pod permission to create and manage executor pods within the same namespace. This approach allows Airflow to submit, monitor, and clean up Spark jobs by simply applying or deleting `SparkApplication` objects, without requiring direct access to a cluster manager.

3.3 Apache Airflow

Apache Airflow is a workflow orchestration platform that allows data engineers to author, schedule, and monitor pipelines as code. A pipeline is expressed as a *Directed Acyclic Graph* (DAG): each node is a *task*, and edges define execution dependencies. Airflow stores DAG definitions, run history, and task states in a relational metadata database and exposes them through a web UI that provides real-time task progress, log access, and manual retry controls.

3.3.1 Architecture

The Airflow architecture in this deployment uses the *KubernetesExecutor*: each task is executed in an isolated Kubernetes pod that is spawned on demand and terminated upon completion. This removes the need for a persistent worker pool, keeps resource usage proportional to actual load, and provides strong task isolation. The Airflow scheduler and web server run as long-lived Kubernetes deployments in the same namespace as the rest of the platform.

3.3.2 Role in the Platform

Airflow enforces the daily batch execution sequence described in Section 2.2. The pipeline DAG submits each Spark application as a `SparkApplication` custom resource via the KubeFlow Spark Operator, monitors the resulting driver pod until completion, and proceeds to the next task only upon success. The DAG is scheduled to run daily after US market close, targeting a 06:00–07:30 UTC window. Airflow version 3.0.2 is deployed from the official Helm chart.

3.4 Apache Iceberg

Apache Iceberg is an open table format designed for large-scale analytic datasets stored in object storage. It separates the *table format* — the layout of metadata and data files — from the *compute engine*, so that Spark, Trino, and other engines can read and write the same tables concurrently without custom connectors. Iceberg’s key capabilities include:

- **ACID transactions.** Writes are committed atomically via a metadata pointer swap. A reader always observes a consistent snapshot even while a concurrent write is in progress.
- **Time-travel queries.** Every commit creates an immutable snapshot. Queries can

be directed at any historical snapshot by specifying a snapshot ID or timestamp, which supports both point-in-time auditing and rollback.

- **Schema and partition evolution.** Columns can be added, renamed, or dropped and partition strategies can be changed without rewriting existing data files or breaking downstream readers.
- **Hidden partitioning.** Partition transforms are tracked in metadata rather than encoded in file paths, so queries do not require partition-column predicates to benefit from partition pruning.

In this platform, all lakehouse data — Bronze raw ingests, Silver cleaned records, and Gold analytical tables — is stored as Iceberg tables. All Spark writes use append-only semantics so that Bronze tables serve as an immutable audit trail of every ingested record. Iceberg version 1.10.1 is used, with the AWS S3 compatibility bundle to communicate with MinIO.

3.5 Apache Gravitino

Apache Gravitino is a vendor-neutral, open-source metadata management server that provides a unified catalog API over heterogeneous data sources. For this platform, Gravitino acts as the *Iceberg REST catalog*: it stores table metadata (snapshot history, schema, partition spec, and property overrides) and exposes the standard Iceberg REST Catalog protocol so that Spark, Trino, and any other Iceberg-compatible engine can discover and access tables through a single endpoint without each engine requiring direct access to the metadata store.

Gravitino is deployed from a custom Helm chart (version 1.1.0) with a PostgreSQL backend for catalog state and is exposed within the cluster through a TLS-secured ingress. OAuth2/OIDC integration with Keycloak ensures that only authenticated services can create or modify catalog objects.

3.6 MinIO

MinIO is a high-performance, S3-compatible object storage server designed for on-premises and private-cloud deployments. It exposes the Amazon S3 API, which means any tool that supports S3 — including Apache Spark, Apache Iceberg, and Trino — can write to and read from MinIO without code changes. MinIO stores data as objects partitioned across distributed erasure-coded sets, providing fault tolerance against disk and node failures.

In this platform, MinIO serves as the physical storage layer for all Iceberg data and metadata files. Spark writes Parquet data files and Iceberg metadata JSON files to MinIO buckets; Trino reads the same files for analytical queries. Separating compute (Spark, Trino) from storage (MinIO) allows each tier to scale independently and keeps storage costs proportional to actual data volume rather than to cluster size.

3.7 Trino

Trino (formerly PrestoSQL) is a distributed SQL query engine optimised for interactive, low-latency analytical queries over large datasets. It operates in a *massively parallel processing* (MPP) model: a *coordinator* node parses and plans a query, decomposes it into stages, and distributes those stages across a set of *worker* nodes that execute them in parallel. Data is exchanged between workers in memory using a pipelined execution model that minimises disk I/O.

Trino connects to data sources through pluggable *connectors*. The Iceberg connector allows Trino to read Iceberg tables stored on MinIO by consulting the Gravitino catalog for metadata and then reading Parquet files directly. This architecture allows analysts to query months of historical OHLCV data, anomaly events, and alert delivery records using standard ANSI SQL without moving data out of the lakehouse.

3.8 PostgreSQL

PostgreSQL is a mature, open-source relational database management system with strong ACID guarantees, a rich type system, and support for advanced features such as window functions, partitioned tables, and logical replication. It is one of the most widely deployed open-source OLTP databases and has a well-documented Kubernetes operator ecosystem.

In this platform, PostgreSQL 17 serves as the operational data store. It maintains the four operational tables: user registrations, custom alert rule definitions, the immutable alert event log, and synchronisation watermarks used by the OLTP–lakehouse bridge job. All application writes go to PostgreSQL first because it offers row-level locking, immediate read-after-write consistency, and simple transactional semantics that are essential for custom alert rule management. The lakehouse receives a delayed, read-only copy of alert events via the nightly sync job, satisfying analytical query requirements without burdening the OLTP store with analytical workloads.

3.9 FastStream

FastStream is an async Python framework for building message-broker-based microservices. It wraps the AIOKafka client with a decorator-driven programming model that mirrors the ergonomics of FastAPI: a developer annotates an async function with the topic it consumes, declares Pydantic models for incoming and outgoing message schemas, and the framework handles serialisation, deserialisation, error handling, and consumer lifecycle management. FastStream also generates an AsyncAPI specification from the broker configuration, providing machine-readable documentation for every topic the service consumes or produces.

FastStream was chosen for all Python microservices in this project — the Rule Engine, Alert Service, Telegram Bot, and the Finnhub producers — because it enforces a consistent asynchronous programming model across the entire codebase, integrates naturally with Pydantic v2 for schema validation, and eliminates the boilerplate of managing AIOKafka consumer loops manually. All Kafka handlers, database calls, and outbound HTTP requests within these services are `async def` coroutines running on Python 3.12 with the `asyncio` event loop.

3.10 Keycloak

Keycloak is an open-source identity and access management server that implements the OAuth 2.0 and OpenID Connect (OIDC) standards. It provides a centralised user directory, a token issuance service, and an administration console for managing clients, roles, and realm-level policies. Keycloak supports multiple authentication flows from the same deployment: the Authorization Code Flow with PKCE is used for browser-based interfaces such as the Airflow web UI, the Gravitino console, and the Trino query editor; client credentials are used for service-to-service calls that run without a human operator present.

When a user or service authenticates, Keycloak issues a signed JSON Web Token (JWT) access token. Each downstream service validates the token locally by fetching the realm's JSON Web Key Set (JWKS) once at startup and verifying the token signature on every request, avoiding a round-trip to Keycloak on the hot path. Keycloak is deployed via its official Helm chart with TLS termination at the ingress layer, and its own state is persisted in a dedicated PostgreSQL database.

3.11 Kubernetes

Kubernetes is an open-source container orchestration system that automates the deployment, scaling, and lifecycle management of containerised workloads. Applications are packaged as OCI container images and described by declarative YAML manifests; the Kubernetes control plane continuously reconciles the observed state of the cluster with the desired state expressed in those manifests. This declarative model makes the full platform reproducible: destroying and recreating the cluster from the manifest files in the repository restores every service to its correct configuration.

Each microservice and infrastructure component in this platform runs as an independent Kubernetes workload (Deployment or StatefulSet), providing strong failure isolation: a crash or OOM kill in one service does not affect other services. Secrets such as API keys and database passwords are stored as Kubernetes Secret objects and mounted as environment variables at pod startup, keeping credentials out of source code and container images. The KubeFlow Spark Operator extends Kubernetes with a `SparkApplication` custom resource, enabling Airflow to submit, monitor, and clean up Spark jobs natively without a separate cluster manager.

Chapter 4

System Design and Deployment

This chapter presents the concrete design of the Stock Anomaly Detection platform and the way each component is deployed on Kubernetes. Section 4.1 walks through the eight functional modules introduced in Chapter ??, detailing the data contracts, control flow, and design decisions of each. Section 4.2 then describes how every module is packaged, configured, and operated on the underlying Kubernetes cluster.

4.1 System Design

4.1.1 Overall Architecture

Figure 4.1.1 presents the overall system architecture. The platform is organised into seven logical layers: *Data Sources*, *Message Queues*, *Data Processing*, *Data Storage*, *Metadata Catalog*, *Security*, and *Serving Layer*. All workloads are deployed as native Kubernetes objects on a shared cluster.

Data Sources. Three independent producers form the inbound edge of the system. A *quote producer* subscribes to the Yahoo Finance WebSocket and publishes consolidated quote events to topic `raw.stock.quotes`. A *trade producer* consumes raw trade ticks from the Finnhub WebSocket and writes them to topic `raw.stock.trades`. A *news producer* polls the Finnhub REST endpoint every five minutes and publishes deduplicated articles to topic `raw.stock.news`. All three producers track the same universe of fifty US-listed symbols.

Message Queues. Apache Kafka, deployed in KRaft mode, serves as the central message bus. It decouples producers from consumers, provides durable buffering for all three

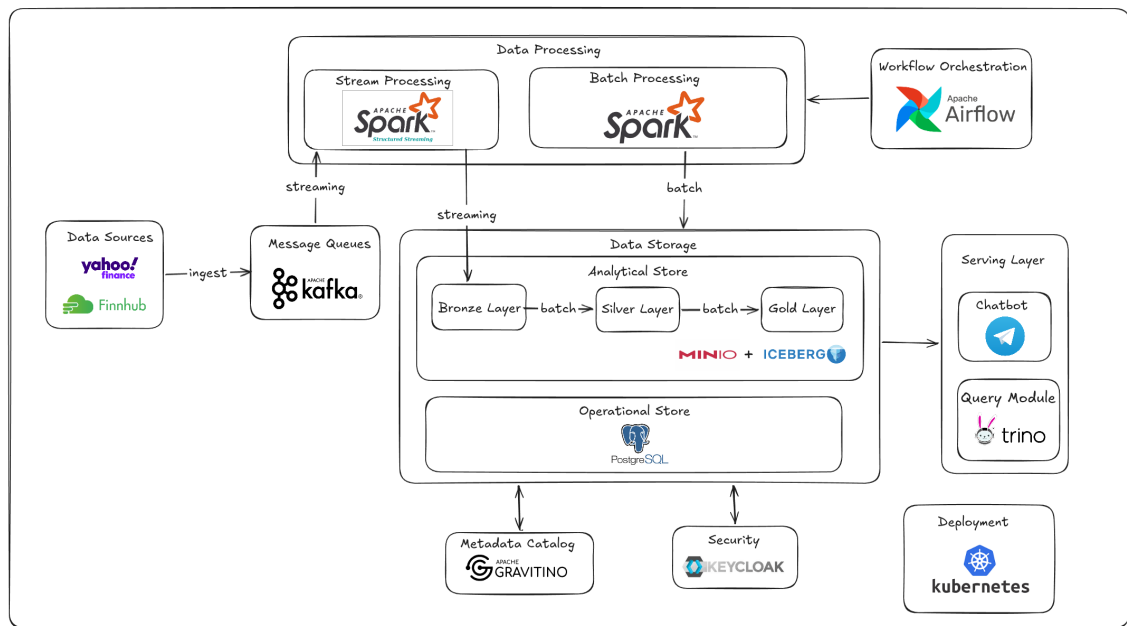


Figure 4.1.1: Overall system architecture.

raw topics, and guarantees per-symbol ordering through a consistent partition-key strategy.

Data Processing. The processing layer is split into two sub-modules, both implemented with Apache Spark. The *Stream Processing* sub-module (Spark Structured Streaming) consumes raw trade and news topics continuously and writes micro-batches into the Bronze layer of the lakehouse. The *Batch Processing* sub-module runs nightly Spark jobs that promote data through the Silver and Gold layers, compute rolling statistics for the rule engine, and maintain the star-schema fact tables for analytical queries.

Workflow Orchestration. Apache Airflow on the *KubernetesExecutor* drives the nightly batch pipeline. Between 06:00 and 07:30 UTC, Airflow submits Spark applications via the KubeFlow Spark Operator: a Yahoo Finance OHLCV loader, a news cleaner, a rule-engine context builder, and a star-schema builder. A synchronisation job promotes custom alert events from PostgreSQL into the Gold layer once per day.

Data Storage. The storage layer contains two stores. The *Analytical Store* is an Apache Iceberg lakehouse on MinIO, organised as Bronze (raw ingest), Silver (cleaned and deduplicated), and Gold (star-schema facts and dimensions) layers. The *Operational Store* is a PostgreSQL database that persists transactional state: user accounts, custom alert rules, alert events, and sync watermarks.

Metadata Catalog. Apache Gravitino acts as the Iceberg REST catalog outside the Data Storage block, storing table metadata — schemas, snapshot histories, and partition

specifications — for all Iceberg tables. Both the Spark jobs and the Trino query engine resolve table locations and schemas through Gravitino, keeping storage and compute cleanly separated.

Security. Keycloak provides OAuth2/OIDC authentication for the platform’s internal HTTP endpoints, including the Gravitino catalog API and the MinIO object-storage console. All service-to-service calls that cross trust boundaries are authenticated through Keycloak-issued tokens.

Serving Layer. Two consumer-facing interfaces are exposed. The *Chatbot* interface delivers real-time anomaly alerts and handles user-defined custom alert commands via the Telegram Bot API; it is backed by the Rule Engine microservice and the Alert Service. The *Query Module* exposes the full lakehouse to interactive SQL clients through Trino, which uses Gravitino as its Iceberg catalog.

The remainder of Section 4.1 describes each module in detail; Section 4.2 covers the Kubernetes deployment for every component.

4.1.2 Data Ingestion Module

The Data Ingestion Module consists of three independent producer services, each responsible for one data source and one Kafka topic. Each producer is an asynchronous Python 3.12 service that encodes messages as UTF-8 JSON using a Pydantic schema and publishes them with the trading symbol as the Kafka partition key, preserving per-symbol ordering for downstream consumers.

Yahoo Finance Quote Producer The quote producer subscribes to the Yahoo Finance WebSocket and publishes consolidated price snapshots to topic `raw.stock.quotes`. Each message is normalised into a `QuoteEvent` model (symbol, price, change percent, day-volume, day-high, day-low, previous close, timestamp) and published with the symbol as the Kafka partition key. The connection is maintained through an exponential-backoff reconnection loop.

Finnhub Trades Producer The trades producer streams raw trade ticks from the Finnhub WebSocket and publishes them to topic `raw.stock.trades`. Each tick is normalised into a `TradeTick` model (symbol, price, volume, timestamp in milliseconds, condition codes). These ticks feed a Spark Structured Streaming job that builds one-minute OHLCV bars. The Kafka producer is configured with `acks="all"` and idempotent delivery to prevent duplicates under broker failover.

Figure 4.1.2: Yahoo Finance quote producer.

Finnhub News Producer The news producer polls the Finnhub /company-news REST endpoint every five minutes and publishes deduplicated articles to topic `raw.stock.news`, which feeds the Bronze layer of the lakehouse. Each article is normalised into a `NewsArticle` model (article ID, symbol, headline, summary, URL, source, category, published timestamp, fetched timestamp). An in-memory MD5-hash set prevents duplicate publication across successive polling windows.

Schema Contract and Operational Visibility Across all three producers, the Pydantic models in each service's `schema.py` module act as the canonical schema for the topic they publish to: the corresponding Spark Scala consumer's `StructType` mirrors these fields exactly, including the distinction between integer and floating-point types and the millisecond-epoch convention for timestamps. Any field rename, addition, or type change must be applied to both sides of the contract simultaneously; in practice, the Python schema is treated as the source of truth and the Scala schema is updated to match.

Operationally, each producer emits a `publish_progress` structured log every fifty (news) or one hundred (quotes, trades) messages, reporting the cumulative published count, the cumulative dropped count, and the most recent symbol. Together with the broker-side view exposed by Kafka UI, this provides an end-to-end signal that ingestion is healthy. Figures 4.1.3 and ?? illustrate two live topics as observed in the Kafka UI.

Offset	Partition	Timestamp	Key	Value
541048	1	5/28/2026, 17:11:42	GOOGL	("symbol":"GOOGL","price":387.92,"change_pct":...
647997	2	5/28/2026, 17:11:42	NVDA	("symbol":"NVDA","price":211.3,"change_pct":-0.6...
558878	0	5/28/2026, 17:11:42	AAPL	("symbol":"AAPL","price":310.96,"change_pct":0.0...
558877	0	5/28/2026, 17:11:41	META	("symbol":"META","price":636.5,"change_pct":0.19...
558876	0	5/28/2026, 17:11:41	AAPL	("symbol":"AAPL","price":310.81,"change_pct":-0.0...
558875	0	5/28/2026, 17:11:41	MSFT	("symbol":"MSFT","price":416.46,"change_pct":0.9...
647996	2	5/28/2026, 17:11:39	NVDA	("symbol":"NVDA","price":211.3,"change_pct":-0.6...
647995	2	5/28/2026, 17:11:37	TSLA	("symbol":"TSLA","price":433.98,"change_pct":-1...
647994	2	5/28/2026, 17:11:37	CSCO	("symbol":"CSCO","price":119.6574,"change_pct":...
647993	2	5/28/2026, 17:11:36	NKE	("symbol":"NKE","price":46.04,"change_pct":0.174...
558874	0	5/28/2026, 17:11:32	META	("symbol":"META","price":636.94,"change_pct":0...
647992	2	5/28/2026, 17:11:32	TSLA	("symbol":"TSLA","price":433.95,"change_pct":-1...
647991	2	5/28/2026, 17:11:30	NVDA	("symbol":"NVDA","price":211.29,"change_pct":...
558873	0	5/28/2026, 17:11:30	V	("symbol":"V","price":328.11,"change_pct":0.13122...
541047	1	5/28/2026, 17:11:29	INTU	("symbol":"INTU","price":312.0,"change_pct":1.387...
558872	0	5/28/2026, 17:11:27	MSFT	("symbol":"MSFT","price":416.6488,"change_pct":...
647990	2	5/28/2026, 17:11:23	NVDA	("symbol":"NVDA","price":211.2465,"change_pct":...
647989	2	5/28/2026, 17:11:23	UNH	("symbol":"UNH","price":382.31,"change_pct":-0.4...
558871	0	5/28/2026, 17:11:22	META	("symbol":"META","price":636.07,"change_pct":0.1...
647988	2	5/28/2026, 17:11:20	NVDA	("symbol":"NVDA","price":211.22,"change_pct":...
558870	0	5/28/2026, 17:11:17	META	("symbol":"META","price":635.44,"change_pct":0...

Figure 4.1.3: Kafka UI: topic `raw.stock.quotes`.

4.1.3 Data Storage Module

The platform maintains two logically separate stores whose access patterns and consistency requirements differ fundamentally. The *Analytical Store* is optimised for high-throughput append writes and large-scale batch reads; the *Operational Store* handles low-latency transactional reads and writes for the alerting subsystem. A daily synchronisation job bridges the two, making operational alert events available for analytical queries without coupling the stores at runtime. Figure 4.1.4 shows the overall storage layout.

Analytical Store The Analytical Store is an Apache Iceberg lakehouse backed by MinIO object storage and organised as three medallion layers.

The *Bronze layer* is the raw-ingest landing zone. Data arrives here directly from Kafka via Spark Structured Streaming (trade ticks, news articles) or from the nightly Yahoo Finance batch loader (daily OHLCV bars). Records in this layer are immutable and never updated; any correction is represented as a new row. The *Silver layer* holds cleaned and deduplicated copies of the Bronze tables. Spark batch jobs remove duplicate articles, enforce consistent schemas, and align timestamps before writing to Silver. The *Gold layer* is the analytics-ready tier. It contains the star-schema fact and dimension tables consumed by Trino and the Rule Engine’s precomputed context table. Writes to this layer are per-

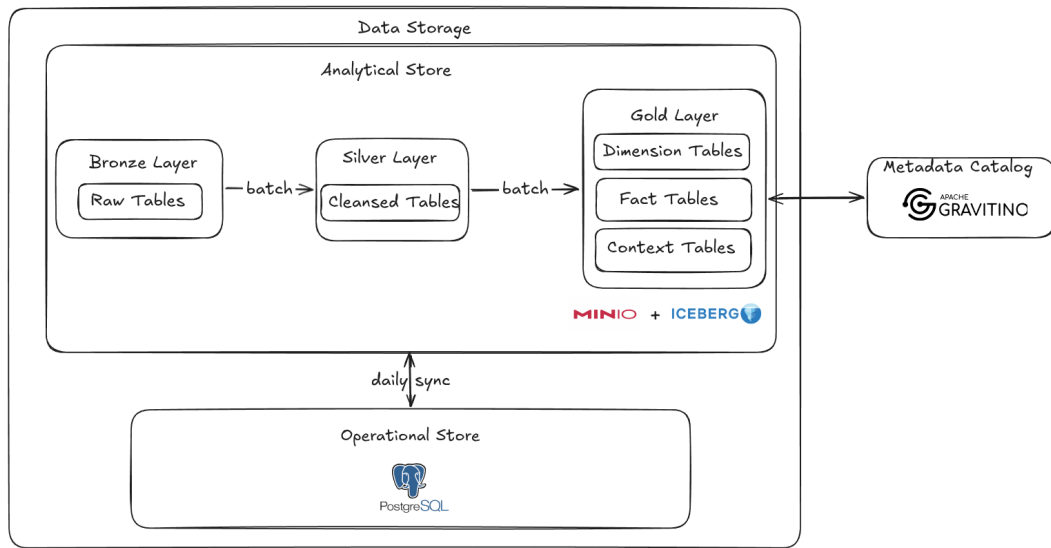


Figure 4.1.4: Data storage module architecture.

formed exclusively by nightly Spark batch jobs; the streaming path never writes to Gold.

Apache Iceberg is chosen as the table format for all three layers because it provides ACID transaction guarantees on object storage, supports schema evolution without rewriting data files, and enables partition pruning that keeps Trino query latency low even as the dataset grows.

Operational Store The Operational Store is a PostgreSQL 15 database that persists the transactional state of the custom alert feature. It contains four tables. `users` maps each Telegram user identifier to an internal UUID. `user_alert_rules` stores the rule definitions — symbols, field, operator, threshold, frequency, and status — created by users via Telegram commands. `user_alert_events` is an append-only event log that records every alert firing with a snapshot of the rule parameters at the time of firing; rows in this table are never updated or deleted. `sync_watermarks` holds the high-water mark used by the OLTP–OLAP sync job to determine which events have already been promoted to the Gold layer.

Metadata Catalog Apache Gravitino serves as the Iceberg REST catalog for the Analytical Store. It stores table metadata — schemas, snapshot histories, and partition specifications — and exposes them through the standard Iceberg REST protocol. Both Spark jobs and Trino resolve table locations through Gravitino at query time, keeping the object storage layout (MinIO) decoupled from the compute engines. Gravitino’s own metadata is persisted in a dedicated PostgreSQL schema and its HTTP API is secured with Keycloak OAuth2 tokens.

4.1.4 Data Processing Module

The Data Processing Module is responsible for transforming raw market data into analytics-ready tables. It consists of two sub-modules that operate on different cadences: *Stream Processing* handles continuous ingest from Kafka, while *Batch Processing* promotes and enriches data through the medallion layers on a nightly schedule.

Stream Processing The Stream Processing sub-module runs two long-lived Spark Structured Streaming jobs. The *trades aggregator* subscribes to `raw.stock.trades`, applies event-time windowing, and aggregates ticks into one-minute OHLCV bars written to `bronze.ohlcvcv_1min`. The *news ingestor* subscribes to `raw.stock.news`, normalises and deduplicates articles, and appends them to `bronze.raw_news_articles`. Both jobs use Iceberg append mode and commit the Kafka offset checkpoint only after each micro-batch write succeeds. Figure 4.1.5 illustrates the common Kafka-to-Iceberg flow shared by both jobs.

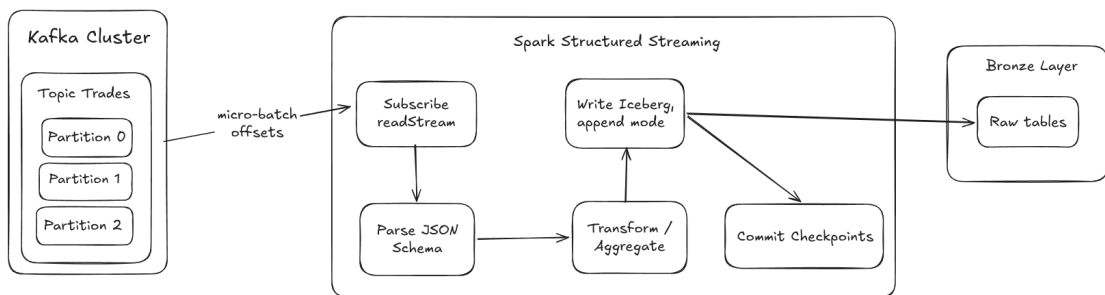


Figure 4.1.5: Kafka-to-Iceberg flow of the stream processing jobs.

Batch Processing The Batch Processing sub-module promotes data through three successive stages, each executed by a dedicated Spark job scheduled via the Workflow Orchestration Module (Section 4.1.5).

In the *Bronze-to-Silver* stage, jobs enforce schema consistency, remove duplicate records, and align timestamps to UTC. The Silver tables are the authoritative cleaned copies consumed by all downstream stages.

In the *Silver-to-Gold fact* stage, Silver OHLCV records are joined against dimension tables and written to the star-schema fact table. Dimension keys are resolved at write time using broadcast joins to avoid shuffle overhead on the large fact dataset.

In the *Silver-to-Gold context* stage, per-symbol aggregations are computed over a twenty-trading-day rolling window and written to `gold.rule_engine_context` — the

sole input to the Rule Engine’s detection logic. All writes to the Gold layer are append-only; corrections are represented as new snapshots.

4.1.5 Workflow Orchestration Module

The Workflow Orchestration Module schedules and coordinates all nightly batch jobs. Apache Airflow, running on the *KubernetesExecutor*, drives the pipeline by submitting *SparkApplication* custom resources to the KubeFlow Spark Operator. Each Spark job runs as an isolated Kubernetes pod pair (driver + executors) and its completion status is tracked through the CRD state. The module defines three DAGs.

Daily OHLCV Pipeline (*spark_ohlc_v_daily_pipeline*, scheduled 0 6 * * *) is the primary pipeline. It chains five Spark jobs through *TimeSensor* gates that enforce minimum start times, ensuring each job has sufficient time to complete before the next depends on its output:

Time (UTC)	Job	Output
06:00	<i>ohlc_v_daily_loader</i>	Fetches daily OHLCV bars from Yahoo Finance; writes to <i>bronze.raw_ohlc_v_daily</i> .
06:30	<i>ohlc_v_daily_cleaner</i>	Cleans and validates Bronze bars; writes to <i>silver.ohlc_v_daily</i> .
07:00	<i>fact_ohlc_v_daily_builder</i>	Builds star-schema fact table <i>gold.fact_ohlc_v_daily</i> from Silver.
07:15	<i>rule_engine_context_builder</i>	Computes 20-day rolling statistics per symbol; writes to <i>gold.rule_engine_context</i> .
07:30	<i>sync_custom_alerts</i>	Appends new <i>user_alert_events</i> rows to <i>gold.fact_alert_history</i> ; advances watermark on success.

Table 4.1.1: Daily OHLCV pipeline job schedule.

Daily News Pipeline (*spark_news_daily_pipeline*, scheduled 0 6 * * *) runs independently and in parallel with the OHLCV pipeline. It executes a single job, *news_cleaner*, which deduplicates raw articles from *bronze.raw_news_articles* into *silver.news_clean*.

Weekly Dimension Pipeline (*spark_batch_weekly_dimension_pipeline*, scheduled 0 5 * * 0, every Sunday) refreshes the slowly-changing dimension tables. It runs *company_info_loader* to fetch current company metadata, followed by *dim_loader* which performs SCD Type 2 updates on *gold.dim_symbol* and rebuilds *gold.dim_date*.

Figure 4.1.6 shows the dependency graph of the daily OHLCV pipeline.

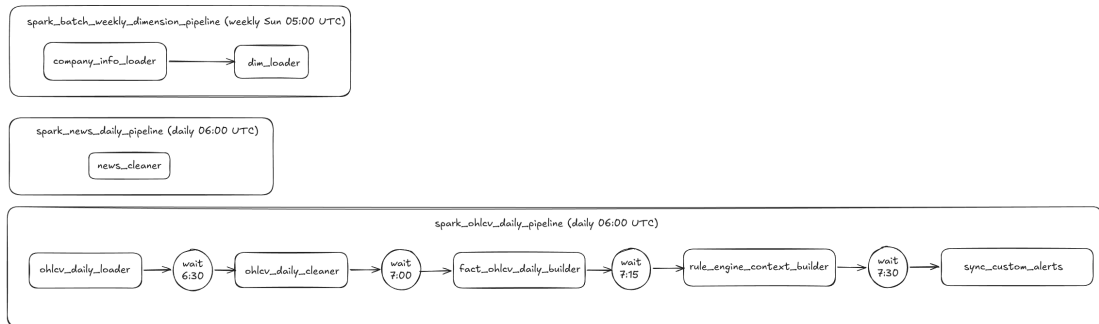


Figure 4.1.6: Daily OHLCV pipeline DAG with TimeSensor gates.

4.1.6 Event Delivery Module

The Event Delivery Module routes anomaly alerts to users over Telegram. It comprises two independent services — the *Alert Service* and the *Telegram Bot* — both backed by the same PostgreSQL user store.

Alert Service The Alert Service consumes `alerts.raw` and fans each alert out to every subscriber of the triggered symbol. Figure 4.1.7 shows the delivery pipeline.

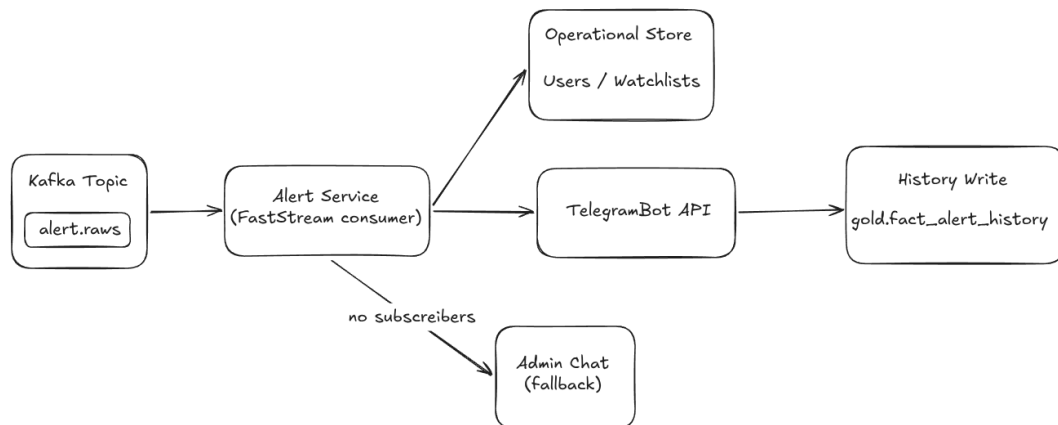


Figure 4.1.7: Alert Service fan-out delivery pipeline.

For each incoming `AlertEvent`, the service resolves the recipient list from a TTL-cached subscriber lookup (backed by PostgreSQL), then dispatches deliveries in parallel via `asyncio.gather`. If no subscribers are found the alert falls back to the admin chat. Before every Telegram send, a two-layer token bucket enforces the bot-wide rate limit (25 msg/s global, 1 msg/s per chat). A successful send is followed by an append

to `gold.fact_alert_history`; any permanent failure is published to the dead-letter topic `alerts.failed` with a typed reason (`RATE_LIMIT`, `PERMANENT`, `TRANSPORT`, or `HISTORY_WRITE`).

The service exposes `POST /internal/reload-subscribers`, called by the Telegram Bot after any `user_preferences` or `user_watchlist` mutation to invalidate the subscriber cache immediately.

Telegram Bot The Telegram Bot handles two concerns: user-facing rule management and delivery of custom-rule firings.

For rule management it exposes the commands `/setalert`, `/listalerts`, `/pausealert`, `/resumealert`, `/resetalert`, `/delalert`, and `/alerthistory`. Rule mutations are persisted to PostgreSQL; after each write that changes the active rule set the bot calls `POST /internal/reload-user-rules` on the Rule Engine to hot-reload the updated rules without a restart.

For custom alert delivery, the Rule Engine sends Telegram notifications directly when a user-defined rule fires, bypassing the Kafka pipeline. The destination chat is resolved from the rule's stored `chat_id`, with a fallback to the admin chat if per-user routing is disabled or the user has not yet run `/start`. Messages for fields derived from the daily batch (`rsi_14`, `bb_position`) include a note that the values reflect end-of-previous-day data.

4.2 System Deployment

All platform components are deployed as native Kubernetes objects on a single-node cluster provisioned with `kubeadm` on Docker Desktop. All workloads share the dedicated namespace `stock-anomaly-detection` to keep cluster resources isolated from other deployments on the same host.

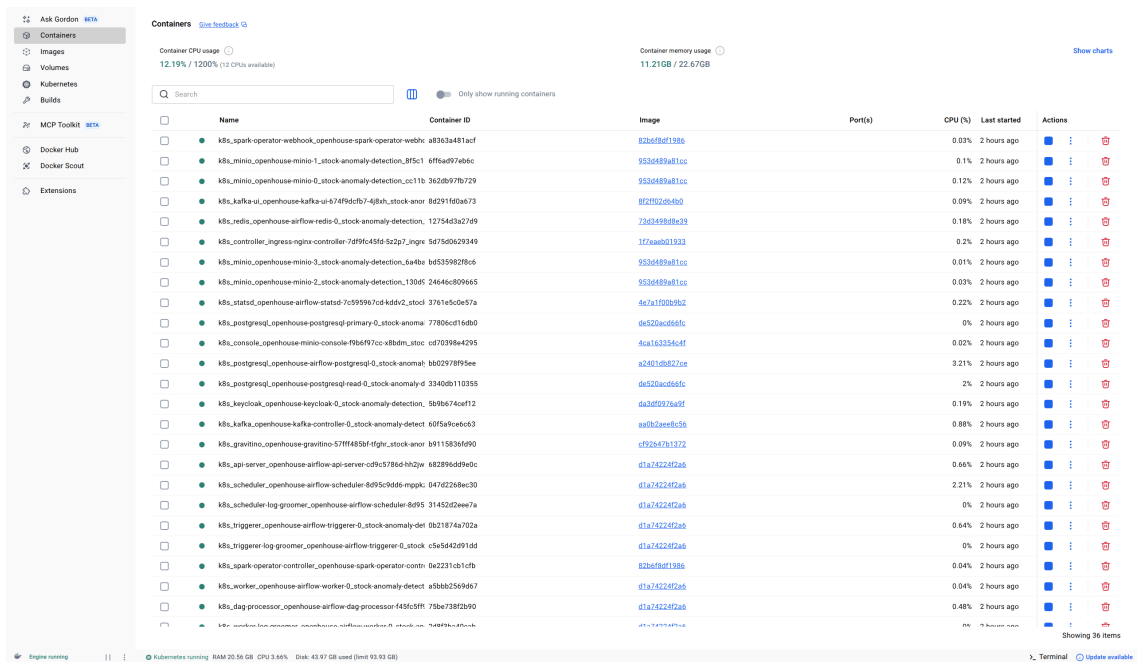


Figure 4.2.1: Single-node Kubernetes cluster running on Docker Desktop.

All services communicate through *ClusterIP* services, which assign a stable internal IP to each pod without exposing any port outside the cluster. This keeps inter-service traffic entirely within the Kubernetes network plane and avoids the need for external load balancers in the local deployment environment.

4.2.1 Data Ingestion Module

Kafka Cluster The message bus is deployed via the Bitnami Kafka 4.0.0 Helm chart running in KRaft mode. Each node runs both the controller and broker roles, configured via `process.roles=broker,controller`, eliminating the need for a separate ZooKeeper ensemble. The quorum is managed entirely within the Kafka pods through the Raft-based KRaft protocol.

The chart creates a StatefulSet named `kafka-controller`. Each pod mounts an 8 GiB PersistentVolumeClaim on the `hostpath` StorageClass for durable log storage. Two Kubernetes Services are provisioned: a *headless service* that exposes the controller-quorum port (9093) and the inter-broker port (9094) for pod-to-pod communication, and a *ClusterIP service* (`openhouse-kafka:9092`) that producer and consumer pods use as their bootstrap address. Figure 4.2.2 illustrates the Kubernetes resource layout.

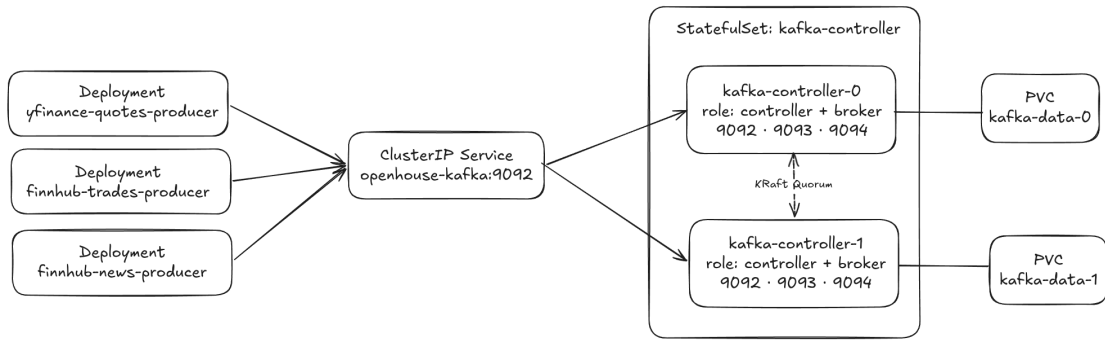


Figure 4.2.2: Kafka KRaft cluster Kubernetes deployment layout.

Producer Deployments Each of the three producer services (Yahoo Finance quote producer, Finnhub trades producer, Finnhub news producer) is packaged as a Docker image and deployed as a single-replica Deployment. Each pod includes an `initContainer` that polls `openhouse-kafka:9092` and blocks until the broker is reachable before the main container starts, ensuring producers never attempt to publish before the cluster is ready.

4.2.2 Data Storage Module

The Data Storage Module comprises three stateful components: an object store for Iceberg data files, a relational database for operational data, and a metadata catalog that connects the two.

Object Store — MinIO MinIO is deployed via its Bitnami Helm chart in *distributed* mode. Four pods form a single erasure-coding set, each mounting a 30 GiB PersistentVolumeClaim on the `hostpath` StorageClass. Distributed mode provides intra-cluster data redundancy even on a single node: a write is committed only after the minimum quorum of shards is confirmed. All Spark jobs and microservices address the store through the ClusterIP service `openhouse-minio:9000` using path-style S3 access.

Relational Store — PostgreSQL PostgreSQL is deployed via the Bitnami chart with the *replication* architecture: one primary pod and one read-replica pod, each with a 20 GiB PVC on `hostpath`. The primary is exposed as `openhouse-postgresql-primary:5432`; write-path services (Rule Engine, Telegram Bot) target the primary directly.

Metadata Catalog — Apache Gravitino Apache Gravitino is deployed as a single-replica Deployment backed by the same PostgreSQL instance. It exposes an Iceberg REST catalog endpoint at `openhouse-gravitino:9001/iceberg/`, which all Spark jobs

and microservices use to resolve table locations and schema evolution. Gravitino maintains the `stock_catalog` namespace that maps every Bronze, Silver, and Gold table to its corresponding MinIO path.

Figure 4.2.3 illustrates the relationships between the three storage components and their consumers.

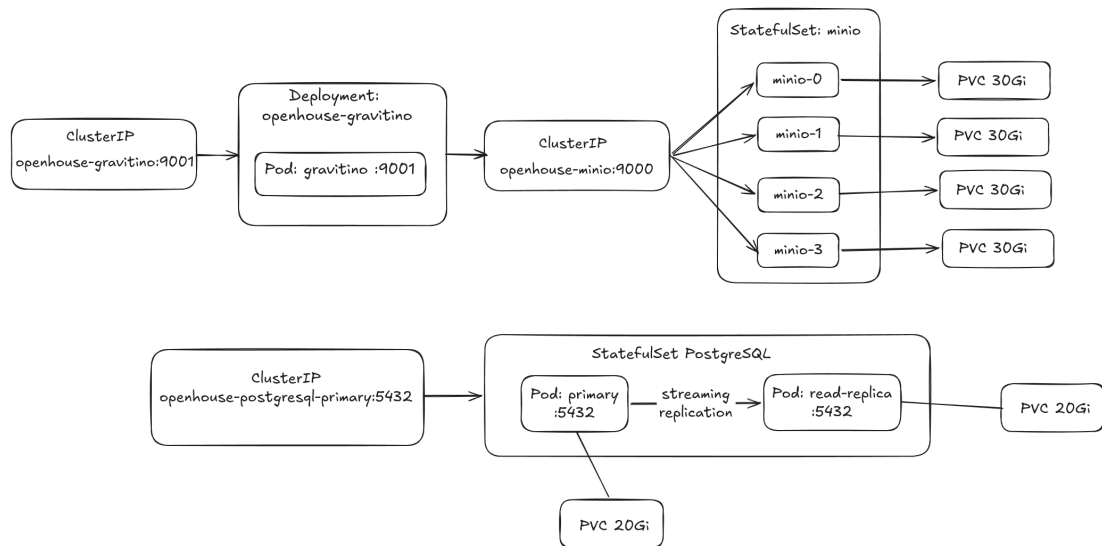


Figure 4.2.3: Data Storage Module Kubernetes deployment layout.

4.2.3 Data Processing Module

All Spark jobs run under the *Kubeflow Spark Operator*, deployed as a single-replica Deployment in the cluster. The operator registers a `SparkApplication` custom resource definition (CRD) and watches for objects of that kind in the `stock-anomaly-detection` namespace. Each batch job is represented as a `SparkApplication` manifest (API version `sparkoperator.k8s.io/v1beta1`, type `Scala`, mode `cluster`).

When Airflow triggers a job, it applies the corresponding `SparkApplication` YAML to the cluster. The operator immediately creates a *driver pod* (1 core, 1 GiB memory) that holds the Spark driver process. The driver then requests the operator to spin up one or more *executor pods* (1 core, 1 GiB memory each). All pods use the `spark` ServiceAccount which is bound to the roles needed to create and manage executor pods. Once the job completes, all pods terminate and the `SparkApplication` object is retained for `timeToLiveSeconds = 3600` before being garbage collected. Figure 4.2.4 illustrates the submission flow.

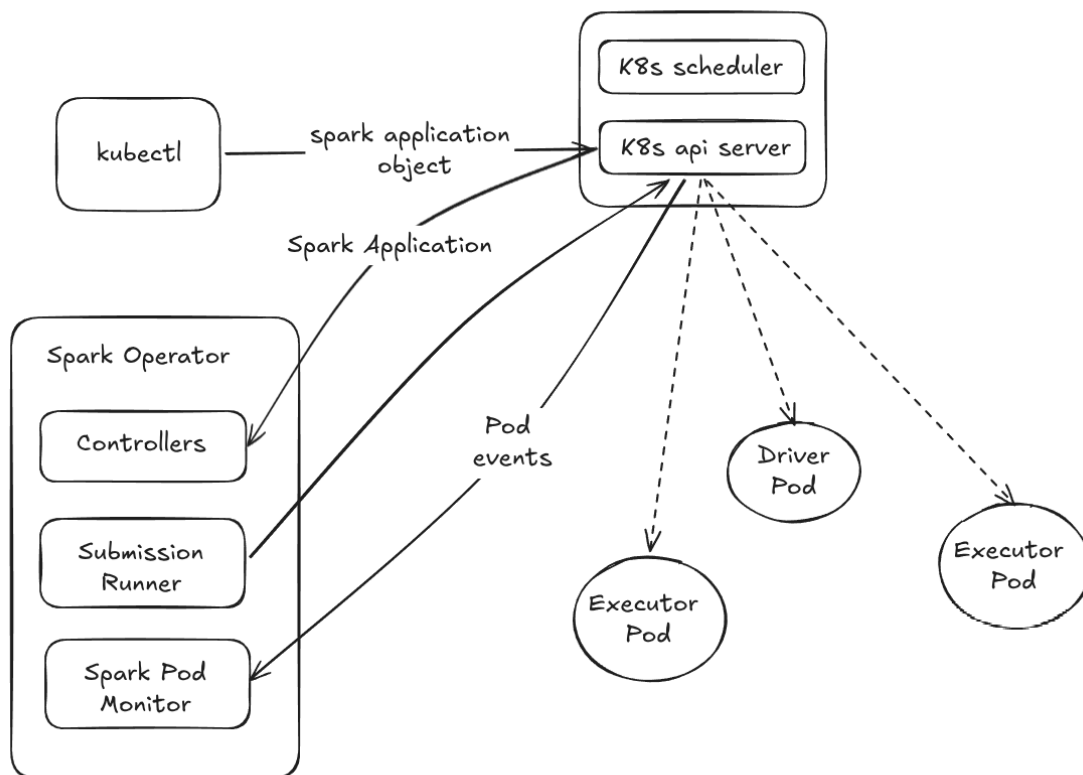


Figure 4.2.4: SparkApplication submission and lifecycle on Kubernetes.

4.2.4 Workflow Orchestration Module

Apache Airflow is deployed via the official Helm chart using the *CeleryExecutor*. The deployment consists of five components: a **scheduler** that parses DAGs and enqueues task instances, a **webserver** that serves the UI, a **triggerer** that manages deferred tasks, a **Celery worker** that executes tasks, and a **Redis** broker that relays task messages between the scheduler and the worker. Airflow's own metadata is stored in a dedicated **PostgreSQL** instance bundled with the chart.

DAG files are delivered to all components through *git-sync*: a sidecar container runs alongside the scheduler, webserver, and worker pods and continuously pulls the DAG repository, making every deployment stateless with respect to DAG code.

Figure 4.2.5 illustrates the component layout.

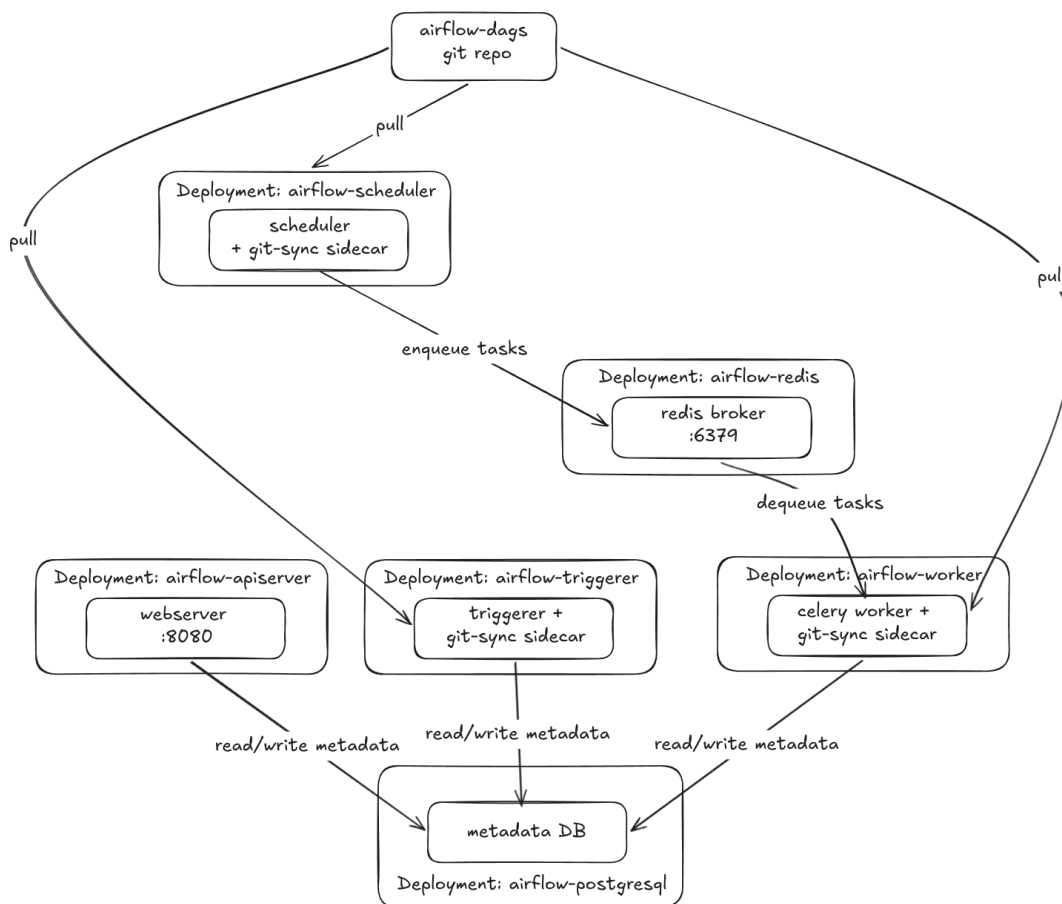


Figure 4.2.5: Airflow CeleryExecutor deployment on Kubernetes.

4.2.5 Event Delivery Module

The Event Delivery Module consists of two single-replica Deployment objects: the **Alert Service** and the **Telegram Bot**.

The Alert Service consumes `alerts.confirmed` from Kafka, fans out messages to registered subscribers, and appends each delivery event to `gold.fact_alert_history` via Gravitino and MinIO. It exposes an internal HTTP endpoint `/internal/reload-subscribers` through a ClusterIP service so that the Telegram Bot can invalidate the subscriber cache after a watchlist change.

The Telegram Bot runs in *webhook* mode. Telegram’s servers push incoming commands to a `/webhook` path exposed outside the cluster via an nginx Ingress backed by an ngrok tunnel. Inside the cluster, the bot is reachable at `telegram-bot:8080`; it writes rule changes to PostgreSQL and calls `rule-engine:8080` to hot-reload user rules after each `/setalert` command.

Figure 4.2.6 illustrates the deployment layout.

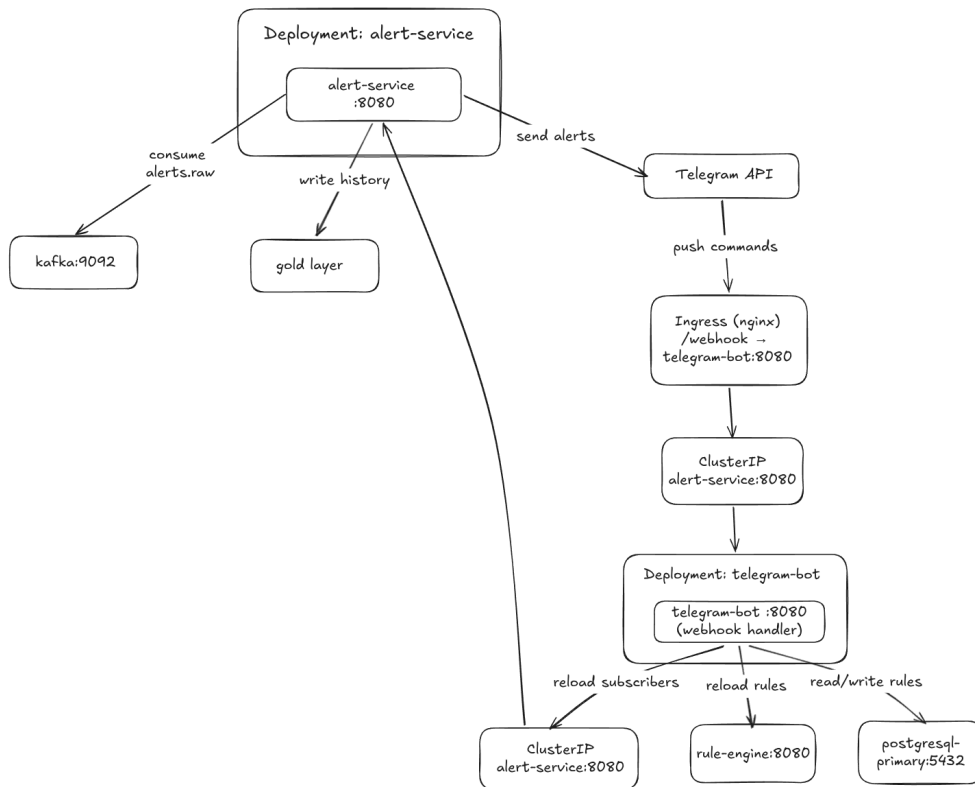


Figure 4.2.6: Event Delivery Module Kubernetes deployment layout.

4.2.6 System Implementation Result

This subsection presents the observed state of the Kubernetes cluster after all manifests and Helm releases have been applied to the stock-anomaly-detection namespace.

Cluster Node. The platform runs on a single-node cluster provisioned with kubeadm on Docker Desktop. The node carries the control-plane role and its status is *Ready*. Figure 4.2.7 shows the node overview.

Pods - docker-desktop									
Nodes - docker-desktop									
Secrets - docker-desktop									
Services - docker-desktop									
Persistent Volume Claims - d...									
1 Item									
Name	CPU	Memory	Disk	Taints	Roles	Version	Age	Conditions	
docker-desktop				0	control-plane	v1.34.1	5d3h	Ready	

Figure 4.2.7: Single control-plane node in the Kubernetes cluster.

Workload Overview. Figure 4.2.8 shows the aggregate workload counts across Pods, Deployments, StatefulSets, and Jobs in the namespace.

Type	Source	Namespace	Involved Object	Message	Count	Age	Last Seen
Normal	controllermanager	stock-anomaly-detect	PodDisruptionBudget	No matching pods found	21	4h10m	4h10m
Warning	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-airflow-tr	Liveness probe failed: No alive jobs found.	1	15m	15m
Warning	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-airflow-tr	Startup probe failed: HTTP probe failed with statuscode: 503	6	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-airflow-tr	Container image "registry.k8s.io/git-sync/git-syncv4.3.0" already present on machine	4	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-airflow-tr	Created container: git-sync	4	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-airflow-tr	Started container git-sync	4	43m	43m
Normal	openhouse-spark-ope	stock-anomaly-detect	Lease: openhouse-spark-s	openhouse-spark-operator-webhook-bfb4d8cb7-25mtt_551b851d-3c46-40b5-8ec4-0336215f9ee became leader	1	42m	42m
Normal	openhouse-spark-ope	stock-anomaly-detect	Lease: openhouse-spark-s	openhouse-spark-operator-controller-d5dd587f-sqwpf_2b4a98a8-8b88-4463-9dc8-60b06eed8647 became leader	1	42m	42m
Warning	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-kafka-con	Liveness probe failed: command timed out: "bgrep -f kafka" timed out after 5s	1	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Container image "ghcr.io/kubeflow/spark-operator/controller:2.5.0" already present on machine	2	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Created container: spark-operator-webhook	2	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Started container spark-operator-webhook	2	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Container image "ghcr.io/kubeflow/spark-operator/controller:2.5.0" already present on machine	3	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Created container: spark-operator-controller	3	43m	43m
Normal	kubelet docker-desk	stock-anomaly-detect	Pod: openhouse-spark-op	Started container spark-operator-controller	3	43m	43m

Figure 4.2.8: Kubernetes workload overview for the stock-anomaly-detection namespace.

Pods. Figure 4.2.9 lists all running pods. The active pods include the Kafka KRaft controller, MinIO distributed nodes, PostgreSQL primary and replica, Gravitino catalog, Spark Operator controller, Airflow components (scheduler, webserver, triggerer, worker, Redis), the three data producers, the Rule Engine, the Alert Service, and the Telegram Bot.

Pods - docker-desktop

Nodes - docker-desktop

Secrets - docker-desktop

Services - docker-desktop

Persistent Volume Claims - d...

ingress-nginx

Search Pods...

29 Items

Name	Namespace	CPU	Memory	Container	Restarts	Controlled By	Node	QoS	Status	Age	
trades-ohlcv-stream-8268049e6a91fef-exec-1	stock-anomaly-di				0	SparkApplication	docker-des	Burstable	Running	65m	
trades-ohlcv-stream-driver	stock-anomaly-di				0	SparkApplication	docker-des	Burstable	Running	65m	
news-ingest-stream-7787c29efe038b7d-exec-1	stock-anomaly-di				0	SparkApplication	docker-des	Burstable	Running	4h5m	
news-ingest-stream-driver	stock-anomaly-di				0	SparkApplication	docker-des	Burstable	Running	4h6m	
finnhub-trades-producer-6775d6869c-q2rdw	stock-anomaly-di				7	ReplicaSet	docker-des	Burstable	Running	4d10h	
finnhub-news-producer-fc9c554-frtpk	stock-anomaly-di				7	ReplicaSet	docker-des	Burstable	Running	4d10h	
yfinance-quotes-producer-59f566d9fb-rjh7b	stock-anomaly-di				7	ReplicaSet	docker-des	Burstable	Running	4d10h	
openhouse-airflow-api-server-cd9c5786d-hh2jw	stock-anomaly-di				7	ReplicaSet	docker-des	BestEffort	Running	4d10h	
openhouse-airflow-dag-processor-f45fc5ff9-2mttt	stock-anomaly-di				59	ReplicaSet	docker-des	Burstable	Running	4d10h	
openhouse-airflow-postgresql-0	stock-anomaly-di				7	StatefulSet	docker-des	Burstable	Running	4d10h	
openhouse-airflow-redis-0	stock-anomaly-di				7	StatefulSet	docker-des	BestEffort	Running	4d10h	
openhouse-airflow-scheduler-8d95c9dd6-mpkiz	stock-anomaly-di				14	ReplicaSet	docker-des	BestEffort	Running	4d10h	
openhouse-airflow-statsd-7c595967cd-kddv2	stock-anomaly-di				7	ReplicaSet	docker-des	BestEffort	Running	4d10h	
openhouse-airflow-triggerer-0	stock-anomaly-di				57	StatefulSet	docker-des	Burstable	Running	4d10h	
openhouse-airflow-worker-0	stock-anomaly-di				57	StatefulSet	docker-des	Burstable	Running	4d10h	
openhouse-kafka-ui-674f9dcfb7-4j8xh	stock-anomaly-di				8	ReplicaSet	docker-des	BestEffort	Running	5d1h	
openhouse-kafka-controller-0	stock-anomaly-di				8	StatefulSet	docker-des	Burstable	Running	5d1h	
openhouse-spark-operator-controller-d5dd587f-sqwpf	stock-anomaly-di				10	ReplicaSet	docker-des	Burstable	Running	5d1h	
openhouse-spark-operator-webhook-bfb4d8cb7-25mtt	stock-anomaly-di				9	ReplicaSet	docker-des	BestEffort	Running	5d1h	
openhouse-gravitino-57fff485bf-ftghr	stock-anomaly-di				8	ReplicaSet	docker-des	Burstable	Running	5d1h	
ingress-nginx-controller-7df9fc45fd-52zp7	ingress-nginx				9	ReplicaSet	docker-des	Burstable	Running	5d2h	
openhouse-keycloak-0	stock-anomaly-di				9	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-postgresql-primary-0	stock-anomaly-di				9	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-postgresql-read-0	stock-anomaly-di				9	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-minio-0	stock-anomaly-di				9	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-minio-1	stock-anomaly-di				10	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-minio-2	stock-anomaly-di				10	StatefulSet	docker-des	Burstable	Running	5d2h	
openhouse-minio-3	stock-anomaly-di				10	StatefulSet	docker-des	Burstable	Running	5d2h	

Figure 4.2.9: Running pods in the stock-anomaly-detection namespace.

Deployments and StatefulSets. Figure 4.2.10 shows all Deployments; stateless services (producers, Rule Engine, Alert Service, Telegram Bot, Airflow components, Spark

Operator) run as Deployments while stateful workloads (Kafka, MinIO, PostgreSQL) run as StatefulSets.

Name	Namespace	Pods	Replicas	Age	Status
ingress-nginx-controller	ingress-nginx	1/1	1	5d2h	Running
finnhub-news-producer	stock-anomaly-detection	1/1	1	4d10h	Running
finnhub-trades-producer	stock-anomaly-detection	1/1	1	4d10h	Running
openhouse-airflow-api-server	stock-anomaly-detection	1/1	1	4d10h	Running
openhouse-airflow-dag-processor	stock-anomaly-detection	1/1	1	4d10h	Running
openhouse-airflow-scheduler	stock-anomaly-detection	1/1	1	4d10h	Running
openhouse-airflow-statsd	stock-anomaly-detection	1/1	1	4d10h	Running
openhouse-gravitino	stock-anomaly-detection	1/1	1	5d1h	Running
openhouse-kafka-ui	stock-anomaly-detection	1/1	1	5d1h	Running
openhouse-minio-console	stock-anomaly-detection	1/1	1	5d2h	Running
openhouse-spark-operator-controller	stock-anomaly-detection	1/1	1	5d1h	Running
openhouse-spark-operator-webhook	stock-anomaly-detection	1/1	1	5d1h	Running
yfinance-quotes-producer	stock-anomaly-detection	1/1	1	4d10h	Running

Figure 4.2.10: Deployments in the stock-anomaly-detection namespace.

Services. All inter-service communication uses ClusterIP services. Figure 4.2.11 lists the active services and their internal cluster addresses.

Name	Namespace	Type	Cluster IP	Ports	External	Selector	Age	Status
openhouse-airflow-postgresql	stock-anomaly-detection	ClusterIP	10.111.120.28	5432:tcp-postgresql/TCP	-	app.kubernetes.io	4d10h	Active
openhouse-airflow-postgresql-hl	stock-anomaly-detection	ClusterIP	None	5432:tcp-postgresql/TCP	-	app.kubernetes.io	4d10h	Active
openhouse-postgresql-primary	stock-anomaly-detection	ClusterIP	10.106.82.182	5432:tcp-postgresql/TCP	-	app.kubernetes.io	5d2h	Active
openhouse-postgresql-primary-hl	stock-anomaly-detection	ClusterIP	None	5432:tcp-postgresql/TCP	-	app.kubernetes.io	5d2h	Active
openhouse-postgresql-read	stock-anomaly-detection	ClusterIP	10.98.196.89	5432:tcp-postgresql/TCP	-	app.kubernetes.io	5d2h	Active
openhouse-postgresql-read-hl	stock-anomaly-detection	ClusterIP	None	5432:tcp-postgresql/TCP	-	app.kubernetes.io	5d2h	Active

Figure 4.2.11: ClusterIP services in the stock-anomaly-detection namespace.

Persistent Volume Claims. Figure 4.2.12 lists all PVCs bound in the namespace. Storage is provisioned on the hostpath StorageClass: Kafka allocates 8 GiB per pod, MinIO 30 GiB per pod, and PostgreSQL 20 GiB per pod (primary and replica each).

Name	Namespace	Storage class	Size	Pods	Age	Status
data-openhouse-airflow-postgresql-0	stock-anomaly-detection	hostpath	8Gi	openhouse	4d10h	Bound
data-openhouse-kafka-controller-0	stock-anomaly-detection	hostpath	8Gi	openhouse	5d1h	Bound
data-openhouse-minio-0	stock-anomaly-detection	hostpath	30Gi	openhouse	5d2h	Bound
data-openhouse-minio-1	stock-anomaly-detection	hostpath	30Gi	openhouse	5d2h	Bound
data-openhouse-minio-2	stock-anomaly-detection	hostpath	30Gi	openhouse	5d2h	Bound
data-openhouse-minio-3	stock-anomaly-detection	hostpath	30Gi	openhouse	5d2h	Bound
data-openhouse-postgresql-primary-0	stock-anomaly-detection	hostpath	20Gi	openhouse	5d2h	Bound
data-openhouse-postgresql-read-0	stock-anomaly-detection	hostpath	20Gi	openhouse	5d2h	Bound
logs-openhouse-airflow-triggerer-0	stock-anomaly-detection	hostpath	30Gi	openhouse	4d10h	Bound
logs-openhouse-airflow-worker-0	stock-anomaly-detection	hostpath	30Gi	openhouse	4d10h	Bound
logs-openhouse-kafka-controller-0	stock-anomaly-detection	hostpath	8Gi	openhouse	5d1h	Bound
redis-db-openhouse-airflow-redis-0	stock-anomaly-detection	hostpath	1Gi	openhouse	4d10h	Bound

Figure 4.2.12: Persistent Volume Claims bound in the stock-anomaly-detection namespace.

Chapter 5

Analytical Problems And Solutions

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 6

Experimental Results

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 7

Conclusion and Future Work

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

Appendix A

This is title of appendix A

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{i=n} x_i = \frac{x_1 + x_2 + \dots + x_n}{n}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\int_0^\infty e^{-\alpha x^2} dx = \frac{1}{2} \sqrt{\int_{-\infty}^\infty e^{-\alpha x^2} dx \int_{-\infty}^\infty e^{-\alpha y^2} dy} = \frac{1}{2} \sqrt{\frac{\pi}{\alpha}}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected

font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\sum_{k=0}^{\infty} a_0 q^k = \lim_{n \rightarrow \infty} \sum_{k=0}^n a_0 q^k = \lim_{n \rightarrow \infty} a_0 \frac{1 - q^{n+1}}{1 - q} = \frac{a_0}{1 - q}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} = \frac{-p \pm \sqrt{p^2 - 4q}}{2}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\frac{\partial^2 \Phi}{\partial x^2} + \frac{\partial^2 \Phi}{\partial y^2} + \frac{\partial^2 \Phi}{\partial z^2} = \frac{1}{c^2} \frac{\partial^2 \Phi}{\partial t^2}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

This is title of appendix B

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\sqrt[n]{a} \cdot \sqrt[n]{b} = \sqrt[n]{ab}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\frac{\sqrt[n]{a}}{\sqrt[n]{b}} = \sqrt[n]{\frac{a}{b}}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain

all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$a \sqrt[n]{b} = \sqrt[n]{a^n b}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{i=n} x_i = \frac{x_1 + x_2 + \dots + x_n}{n}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

$$\int_0^\infty e^{-\alpha x^2} dx = \frac{1}{2} \sqrt{\int_{-\infty}^\infty e^{-\alpha x^2} dx \int_{-\infty}^\infty e^{-\alpha y^2} dy} = \frac{1}{2} \sqrt{\frac{\pi}{\alpha}}$$

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.